Dhirubhai Ambani University

IT305 LAB – Socket Programming Project

Fault-Tolerant Topic-based File Distribution Service

Design Document

Project Focus

TCP Socket Programming | Multi-threaded Servers | Fault Tolerance
Session Management | Performance Measurement

Project Groups	Group 097 and Group 112
Submission	Design Document
Design Deadline	27 September 2026

Group Members

No.	Full Name	Student ID
1	Aryan Dalsaniya	202401405
2	Harshil Shah	202401468
3	Dhvanit Shah	202401467
4	Dwij Patel	202401418
5	Tanishq Shah	202401474
6	Niranjan Panchal	202401441

Contents

1 Project Overview **3**
1.1 Objectives . . . 3
1.2 Scope . . . 3

2 Requirements and Design Assumptions **3**

3 High-Level System Architecture **4**

4 Server Data Structures **4**
4.1 Topic-to-Directory Mapping . . . 4
4.2 Session Checkpoint State . . . 5

5 Part I – Basic Topic-Based File Distribution **5**
5.1 Part I Workflow . . . 5
5.2 Part I Message Format . . . 6
5.3 Single-Threaded Server . . . 6
5.4 Multi-Threaded Server . . . 6

6 Part I Performance Measurement **7**

7 Part II – Fault Injection **7**
7.1 Failure Probability . . . 7

8 Part II – Case 1: No Session Management **8**
8.1 Design . . . 8
8.2 Case 1 Message Flow . . . 8

9 Part II – Case 2: Session Management and Resume **8**
9.1 Checkpoint Design . . . 9
9.2 Resume Protocol . . . 9
9.3 Case 2 Messages . . . 10
9.4 Checkpoint Consistency . . . 10

10 Part II – Case 2 Enhanced **10**
10.1 Parallel Connection Design . . . 10
10.2 Enhanced Protocol . . . 11
10.3 Failure Recovery . . . 11
10.4 Enhanced Performance Measurements . . . 11

11 Sequence Diagrams **12**
11.1 Part I Sequence . . . 12
11.2 Case 2 Resume Sequence . . . 13

12 Module and File Structure **13**

12.1 Key Functions 14

13 Concurrency and Synchronization **15**
13.1 Resource Management 15

14 Performance and Result Collection Plan **16**
14.1 Part I Metrics 16
14.2 Part II Metrics 16
14.3 Planned Result Tables 16

15 Extra Features **17**

16 Design Alternatives and Rationale **18**
16.1 Thread-per-client versus a shared worker pool 18
16.2 Blocking versus non-blocking sockets 18
16.3 Checkpoint Granularity 18
16.4 Parallel TCP Connections 18

17 Error Handling and Edge Cases **18**

18 Testing Strategy **19**

19 Planned Project Workflow **19**

20 Conclusion **20**

1 Project Overview

The project implements a fault-tolerant, topic-based file distribution service using TCP sockets. The system consists of a server that maintains a mapping between topic names and directories and one or more clients that request all files belonging to a selected topic.

The project is divided into Part I and Part II. Part I implements basic topic-based file distribution using both a single-threaded server and a multi-threaded server. Part II extends the multi-threaded implementation with fault injection and two different approaches to handling interrupted transfers: a restart-from-beginning approach without session management and a checkpoint-based resume approach with session management.

The server receives the root directory containing topic sub-directories through its command-line arguments. Therefore, dataset paths are not hard-coded into the implementation.

1.1 Objectives

The main objectives are:

1. Implement a TCP-based client-server file distribution system.
2. Represent topics and their corresponding directories using an appropriate server-side data structure.
3. Implement both sequential and concurrent request handling.
4. Measure and compare transfer performance for single-threaded and multi-threaded servers.
5. Introduce probabilistic connection failures during file transfer.
6. Implement a no-session-management case in which an interrupted transfer restarts.
7. Implement session management using checkpoints so that an interrupted transfer can resume.
8. Design an enhanced version with configurable parallel TCP connections.
9. Record performance measurements for comparison across cases.

1.2 Scope

The system is designed for a Linux/POSIX environment and uses TCP sockets and POSIX threads. The required command-line interfaces are preserved so that the implementation can be tested using the course's automated testing environment.

2 Requirements and Design Assumptions

The server and client will receive all environment-specific information through command-line arguments. No IP address, port number, or dataset path will be hard-coded.

Table 1: Required command-line interface

Program	Required invocation
Part I server	./server <port> <topics_root_dir>
Part II server	./server <port> <topics_root_dir> <failure probability>
Client	./client <server_ip> <server_port> <topic> <output_dir>

The topic root directory contains one sub-directory per topic. The sub-directory name acts as the topic key.

3 High-Level System Architecture

The system has two primary components: a server and a client.

- **Server:** creates a listening TCP socket, accepts clients, resolves requested topics, reads files from the corresponding directory, and sends file data according to the selected project case.
- **Client:** establishes a TCP connection, requests a topic, receives the files, creates the required output directory, and stores the received files.

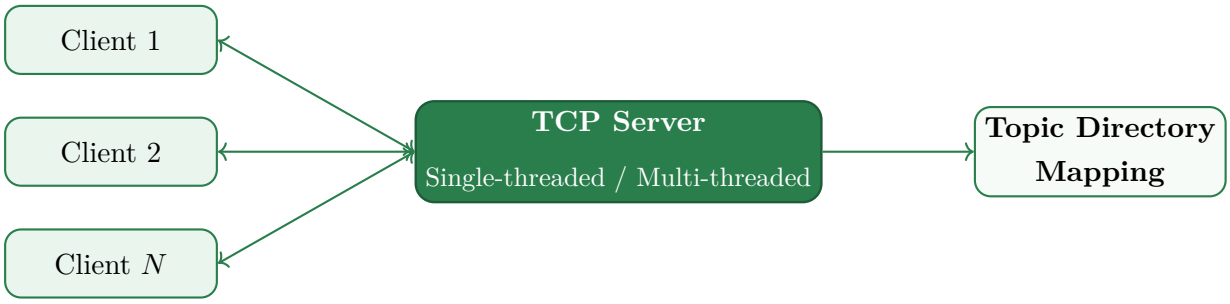


Figure 1: High-level system architecture

In the multi-threaded implementation, each accepted client connection is handled by a separate POSIX thread. This provides concurrent service to multiple clients.

4 Server Data Structures

4.1 Topic-to-Directory Mapping

The server maps each topic key to the directory containing its files. The mapping is constructed dynamically from the supplied `topics_root_dir`.

Each entry contains:

- topic name;
- corresponding directory path.

Conceptually:

```
struct TopicEntry {
    char topic[MAX_TOPIC_LEN];
    char directory[MAX_PATH_LEN];
};
```

A dynamically allocated collection of topic entries will be used. Since the number of topics is expected to be small, a linear lookup is sufficient.

4.2 Session Checkpoint State

For Part II Case 2, the system maintains checkpoint information containing:

- topic name;
- current file name;
- byte offset within the current file;
- transfer/session identifier;
- completion status.

The checkpoint represents the latest progress that has been successfully acknowledged by the client.

5 Part I – Basic Topic-Based File Distribution

5.1 Part I Workflow

The basic request flow is:

1. The server creates a TCP listening socket and binds it to the supplied port.
2. The server accepts a client connection.
3. The client sends a request in the form `GET <topic>`.
4. The server looks up the topic in its topic-to-directory mapping.
5. If the topic exists, the server enumerates the files in the directory.
6. The server sends file metadata followed by the corresponding file data.

7. The client creates `output_dir/topic/` and stores the received files.

8. After all files have been transferred, the server sends `TRANSFER_END`.

9. The connection is closed.

5.2 Part I Message Format

The protocol uses explicit framed control messages. File data is transmitted as binary payloads whose length is specified by the preceding file metadata.

Table 2: Part I control messages

Message	Purpose
`GET <topic>`	Client requests all files belonging to a topic.
`FILE <name> <size>`	Server announces a file name and the number of bytes that follow.
`FILE_END`	Indicates completion of the current file.
`TRANSFER_END`	Indicates completion of the complete topic transfer.
`ERROR <code>`	Indicates an invalid topic or protocol error.

The receiver uses the announced file size to determine exactly how many binary bytes belong to the file. Control messages are framed separately from binary payloads so that TCP stream boundaries do not affect message interpretation.

5.3 Single-Threaded Server

The single-threaded server processes clients sequentially. After accepting a client, the server handles the complete request before accepting the next client.

```
while (server_is_running) {
    client_fd = accept(listen_fd, ...);
    handle_client(client_fd);
    close(client_fd);
}
```

This implementation provides the baseline for performance comparison.

5.4 Multi-Threaded Server

The multi-threaded server creates one POSIX thread for each accepted client.

```
while (server_is_running) {
    client_fd = accept(listen_fd, ...);
    create_client_thread(client_fd);
}
```

Client-specific state is stored in the corresponding thread context. The topic mapping is initialized before worker threads are created and is treated as read-only during normal operation.

6 Part I Performance Measurement

For each transfer, the implementation will measure:

- elapsed transfer time;
- total data bytes transmitted;
- aggregate transfer rate.

The aggregate transfer rate is:

$$\text{Transfer Rate} = \frac{\text{Total Data Bytes Sent}}{\text{Elapsed Transfer Time}}.$$

A high-resolution POSIX timing mechanism such as `clock_gettime()` will be used.

Experiments will vary the number of clients and compare the single-threaded and multi-threaded servers. Results will be stored in CSV files and represented using tables and graphs.

7 Part II – Fault Injection

Part II requires the multi-threaded server to remain correct when a connection is interrupted during an active transfer.

7.1 Failure Probability

The server receives a failure probability as a command-line argument. This value controls the probability with which the server intentionally terminates an active client connection.

The planned mechanism is:

1. Initialize a pseudorandom number generator at server startup.
2. During an active transfer, after a defined transfer unit, generate a random value.
3. Compare the generated value with the configured failure probability.
4. If the failure condition is met, terminate the active client connection.

A failure probability of zero disables intentional failures.

The failure decision point will remain consistent across experiments so that different implementations can be compared under the same testing conditions.

8 Part II – Case 1: No Session Management

Case 1 does not retain metadata about an interrupted transfer.

8.1 Design

The client initially sends:

```
GET <topic>
```

The server starts transmitting the requested topic from the first file.

If the connection fails, no checkpoint is used. The client reconnects and sends the same:

```
GET <topic>
```

request again.

The server therefore starts the complete topic transfer from the beginning.

8.2 Case 1 Message Flow

1. Client connects to server.
2. Client sends `GET <topic>`.
3. Server sends files sequentially.
4. Fault injection may terminate the connection.
5. Client detects the connection failure.
6. Client establishes a new TCP connection.
7. Client sends `GET <topic>` again.
8. Server retransmits the topic from the beginning.

If a failure occurs while transferring file F_k, the subsequent connection starts from F_1. Therefore, some data may be transmitted more than once.

This case provides the baseline for evaluating session management.

9 Part II – Case 2: Session Management and Resume

Case 2 maintains state about an in-progress transfer and uses checkpoints to resume after a connection failure.

9.1 Checkpoint Design

A checkpoint contains:

- topic;
- current file name;
- byte offset already transferred and acknowledged;
- transfer/session identifier.

The checkpoint is updated after a defined data chunk has been successfully received and recorded by the client.

9.2 Resume Protocol

The planned sequence is:

1. Client requests a topic.
2. Server sends file metadata.
3. Server sends file data in chunks.
4. Client writes the received data.
5. Client sends an acknowledgement containing the latest successfully recorded checkpoint.
6. Server records the acknowledged checkpoint.
7. If the connection fails, the client reconnects.
8. Client sends its latest checkpoint using a `RESUME` message.
9. Server validates the checkpoint.
10. Server resumes transmission from the recorded byte offset.
11. The transfer continues until all files are complete.

9.3 Case 2 Messages

Table 3: Case 2 protocol messages

Message	Purpose
`GET <topic>`	Client requests a topic transfer.
`RESUME <session> <topic> <file> <offset>`	Reports the latest client checkpoint after reconnecting.
`FILE <name> <size>`	Server announces the file being transferred.
`DATA <offset> <length>`	Identifies the byte range carried by a data chunk.
`ACK <file> <offset>`	Confirms that data up to the specified checkpoint has been recorded.
`FILE_END`	Indicates completion of the current file.
`TRANSFER_END`	Indicates completion of the complete topic transfer.
`ERROR <code>`	Indicates an invalid topic, checkpoint, or protocol request.

The exact framing and integer representation will be implemented consistently by both client and server.

9.4 Checkpoint Consistency

The client acknowledgement acts as the checkpoint commit point. The server does not advance its recorded checkpoint until it receives the corresponding acknowledgement.

Consequently, after a connection failure, only data transmitted after the latest acknowledged checkpoint may need to be retransmitted.

10 Part II – Case 2 Enhanced

The enhanced implementation will use parallel TCP connections as the primary performance enhancement.

Blocking sockets will be used because they provide a simpler control flow while parallelism is achieved through multiple simultaneous connections.

10.1 Parallel Connection Design

A configurable number K of TCP connections will be used for a single topic transfer.

The files belonging to a topic will be distributed among the available connections. Each connection will independently transfer its assigned files.

The client will maintain the output files according to their file names and transfer offsets.

10.2 Enhanced Protocol

The enhanced implementation uses the Case 2 checkpoint protocol on each connection.

Each connection maintains its own logical transfer state:

- connection/session identifier;
- assigned file;
- current byte offset;
- latest acknowledged checkpoint.

The messages used by each connection are:

Table 4: Case 2 Enhanced protocol

Message	Purpose
`GET <topic>`	Initial topic request.
`RESUME <session> <topic> <file> <offset>`	Resume an interrupted connection from its checkpoint.
`FILE <name> <size>`	Assign and announce a file.
`DATA <offset> <length>`	Transfer a chunk of file data.
`ACK <file> <offset>`	Confirm successful recording of a checkpoint.
`FILE_END`	Indicate completion of an assigned file.
`TRANSFER_END`	Indicate completion of the overall topic transfer.
`ERROR <code>`	Indicate a protocol or checkpoint error.

10.3 Failure Recovery

Each parallel connection maintains its own progress. If one connection fails, only its unfinished work needs to be resumed. Other active connections may continue transferring their assigned work.

Concurrent writes to the same file will be avoided unless non-overlapping byte ranges are explicitly assigned.

10.4 Enhanced Performance Measurements

The enhanced implementation will measure:

- total completion time;
- aggregate throughput;

- useful bytes delivered;
- redundant bytes transmitted;
- effect of the number of parallel connections.

The number of parallel connections will have a sensible default so that the required command-line interface remains valid.

11 Sequence Diagrams

11.1 Part I Sequence

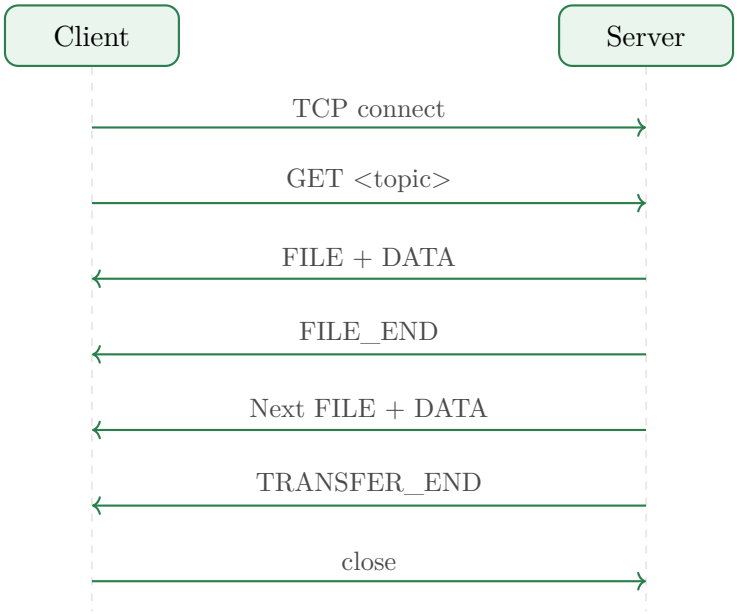


Figure 2: Part I client-server sequence

11.2 Case 2 Resume Sequence

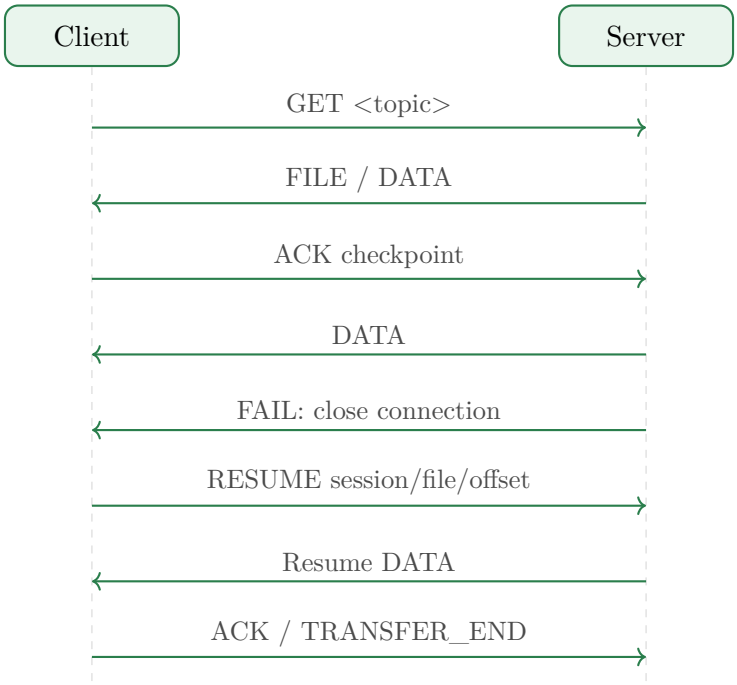


Figure 3: Case 2 failure and resume sequence

12 Module and File Structure

The final submission contains separate folders for Part 1, Part 2 Case 1, Part 2 Case 2, and Part 2 Case 2 Enhanced.

The planned structure is:

```
FinalSubmission_Group_097_112/
|
+-- FinalReport_Group_097_112.pdf
+-- README.md
|
+-- Part1/
|   +-- server.c
|   +-- client.c
|   +-- protocol.h
|   +-- common.c
|   +-- common.h
|   +-- Makefile
|
+-- Part2_Case1/
|   +-- server.c
|   +-- client.c
|   +-- protocol.h
|   +-- common.c
```

```
|   +-- common.h
|   +-- Makefile
|
+-- Part2_Case2/
|   +-- server.c
|   +-- client.c
|   +-- protocol.h
|   +-- session.c
|   +-- session.h
|   +-- common.c
|   +-- common.h
|   +-- Makefile
|
+-- Part2_Case2_Enhanced/
|   +-- server.c
|   +-- client.c
|   +-- protocol.h
|   +-- session.c
|   +-- session.h
|   +-- common.c
|   +-- common.h
|   +-- Makefile
|
+-- Results/
```

12.1 Key Functions

The following are planned logical functions. Function names may be adjusted during implementation without changing the architecture.

Table 5: Planned key functions

Function	Responsibility
`create_server_socket()`	Create, bind, and listen on the TCP socket.
`accept_client()`	Accept a new client connection.
`handle_client()`	Process one client's request.
`load_topics()`	Construct the topic-to-directory mapping.
`find_topic()`	Resolve a topic to its directory.
`send_file()`	Transfer one file to a client.
`receive_file()`	Receive and store one file.
`send_all_files()`	Transfer every file in a topic directory.
`inject_failure()`	Apply the configured fault-injection policy.
`save_checkpoint()`	Record acknowledged transfer progress.
`load_checkpoint()`	Recover checkpoint state.
`resume_transfer()`	Continue a transfer from a validated checkpoint.
`measure_transfer()`	Record timing and byte counters.

13 Concurrency and Synchronization

The multi-threaded server will use POSIX threads. Each client connection will normally have a dedicated worker thread.

The topic mapping is initialized before worker threads are created and is treated as read-only during normal operation. Client-specific transfer state is local to the worker thread.

Shared resources that genuinely require protection, such as centralized logging or shared session/checkpoint tables, will use a POSIX mutex.

13.1 Resource Management

For every socket, file descriptor, dynamically allocated buffer, and thread resource, the implementation will define an ownership and cleanup path.

In particular:

- close client sockets after a transfer or failure;
- close opened files after transmission;
- free dynamically allocated memory;
- properly manage worker threads;

- handle failed system calls without leaking resources.

14 Performance and Result Collection Plan

The final implementation will generate raw measurements that can be processed into tables and graphs.

14.1 Part I Metrics

For different numbers of concurrent clients, the following will be measured:

- elapsed transfer time;
- bytes transmitted;
- aggregate transfer rate.

The principal comparison will be between the single-threaded and multi-threaded servers.

14.2 Part II Metrics

For different fault-injection probabilities, the following will be recorded:

- total bytes transmitted;
- useful bytes successfully delivered;
- redundant bytes retransmitted;
- completion time;
- throughput;
- session-management overhead.

For the enhanced case, the effect of the number of parallel TCP connections will also be measured.

14.3 Planned Result Tables

The final report will contain tables similar to the following.

Table 6: Part I performance comparison template

Clients	Mode	Time (s)	Bytes	Rate (bytes/s)
1	Single	–	–	–
1	Multi	–	–	–
2	Single	–	–	–
2	Multi	–	–	–
4	Single	–	–	–
4	Multi	–	–	–

Table 7: Part II performance comparison template

Failure Prob.	Case	Total Bytes	Useful Bytes	Redundant Bytes	Time
0	Case 1	–	–	–	–
0.1	Case 1	–	–	–	–
0.1	Case 2	–	–	–	–
0.3	Case 1	–	–	–	–
0.3	Case 2	–	–	–	–

No numerical performance claim is made in this design document because actual measurements will only be available after implementation and experiments.

15 Extra Features

The following additional features are planned where they can be implemented without interfering with the required interface:

1. **Logging:** record connection events, requested topics, transfer progress, failures, and completion information.
2. **Configurable parameters:** retain the required command-line arguments and use optional trailing arguments where useful.
3. **Graceful cleanup:** ensure sockets and files are closed when a transfer finishes or fails.
4. **Extensible topic discovery:** construct the topic mapping from the supplied root directory rather than hard-coding topic names.

These features improve maintainability, debugging, and portability without changing the required command-line interface.

16 Design Alternatives and Rationale

16.1 Thread-per-client versus a shared worker pool

A thread-per-client design is selected because the project explicitly focuses on multi-threaded server programming and provides a straightforward implementation for concurrent client handling.

A thread pool could reduce thread-creation overhead, but it introduces additional queue management and synchronization complexity.

16.2 Blocking versus non-blocking sockets

Blocking sockets are selected for the initial implementation because they provide a simpler control flow. Concurrency is achieved by assigning clients to separate threads.

Non-blocking sockets could be explored as an alternative, but combining non-blocking I/O with session management and parallel connections would increase implementation complexity.

16.3 Checkpoint Granularity

Checkpointing after every data chunk provides a clear resume point and limits the amount of data that may need retransmission after a failure.

Checkpointing only after every file would reduce acknowledgement overhead but could cause more redundant transmission when a failure occurs during a large file.

16.4 Parallel TCP Connections

Parallel TCP connections are selected as the enhanced feature because they provide additional concurrency for transferring the files of one topic.

The number of connections remains configurable so that the performance effect can be measured experimentally.

17 Error Handling and Edge Cases

The implementation will account for:

- requested topic does not exist;
- topic directory cannot be opened;
- file cannot be opened;
- client disconnects unexpectedly;
- server experiences a failed socket operation;
- invalid or inconsistent resume checkpoint;

- zero failure probability;
- failure near a file boundary;
- failure during a file transfer;
- empty topic directory;
- multiple clients requesting the same topic concurrently.

For an invalid checkpoint, the server will reject the resume request rather than silently interpreting an invalid offset.

18 Testing Strategy

Testing will be performed incrementally:

1. Verify basic TCP connection establishment.
2. Verify a valid topic request and transfer of a single file.
3. Verify transfer of all files in a topic.
4. Verify invalid-topic handling.
5. Verify multiple simultaneous clients.
6. Compare single-threaded and multi-threaded behavior.
7. Run Part II with failure probability zero.
8. Run fault-injection experiments with increasing failure probabilities.
9. Verify Case 1 restarts from the beginning.
10. Verify Case 2 resumes from the recorded checkpoint.
11. Verify that completed files are not unnecessarily retransmitted in the session-managed case.
12. Test the enhanced implementation with different numbers of parallel connections.
13. Verify that the required command-line invocations work without modification.

19 Planned Project Workflow

The implementation will proceed in the following order:

1. Build and test the common TCP client/server communication layer.
2. Implement topic discovery and file enumeration.

3. Implement Part I single-threaded server.
4. Implement the Part I client.
5. Add the multi-threaded server.
6. Add timing and byte-count instrumentation.
7. Implement Part II Case 1 fault injection.
8. Implement Part II Case 2 checkpoints and resume.
9. Validate failure recovery under repeated fault-injection tests.
10. Implement Case 2 Enhanced parallel TCP connections.
11. Run controlled performance experiments.
12. Generate CSV logs and graphs.
13. Prepare the final report and README.

20 Conclusion

This design provides a progressive implementation path from a basic TCP topic-based file server to a concurrent and fault-tolerant file distribution system.

Part I establishes the basic transfer mechanism and provides a baseline for performance comparison. Part II introduces controlled connection failures, followed first by restart-based recovery and then by checkpoint-based session management. The enhanced design adds configurable parallel TCP connections to investigate further performance improvements.

The design separates protocol handling, file transfer, session state, concurrency, fault injection, and performance measurement so that the individual components can be implemented and tested independently.